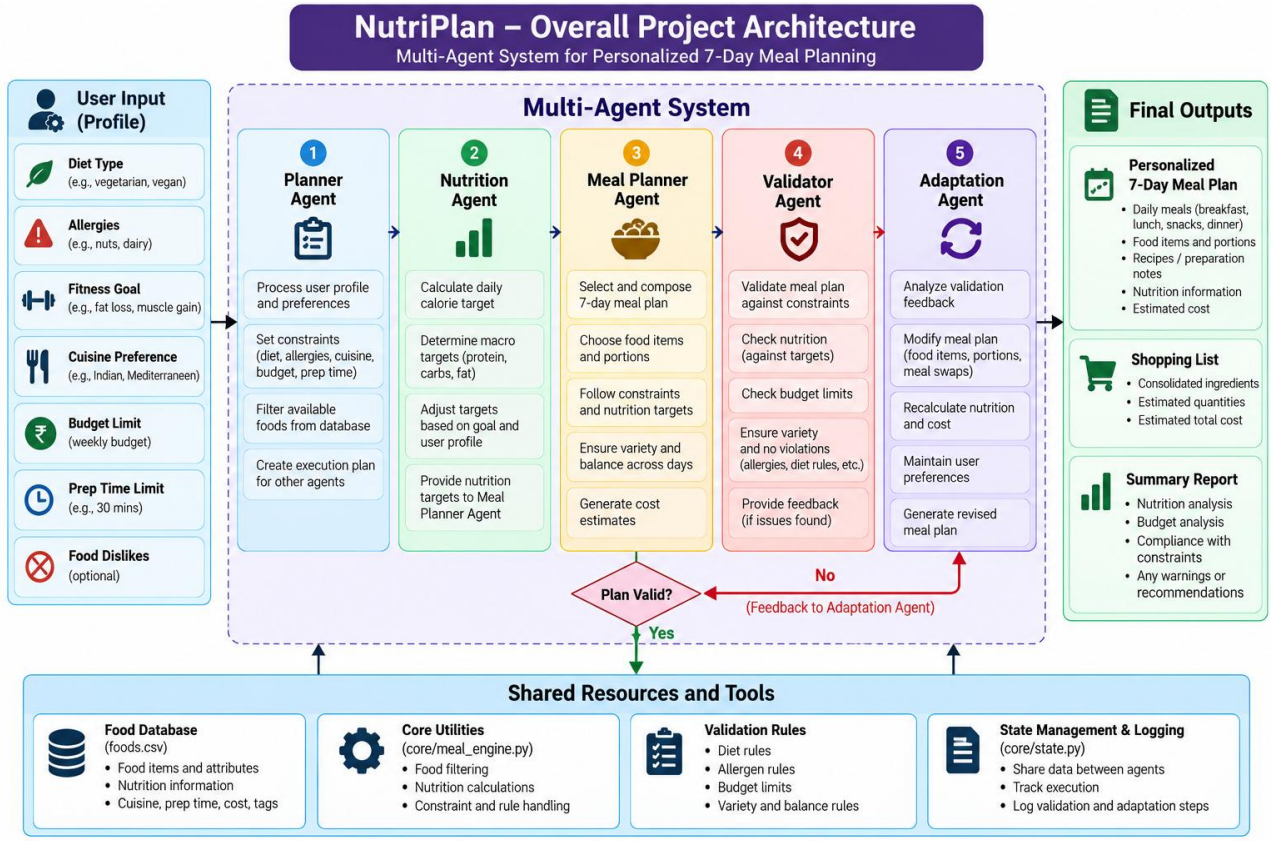
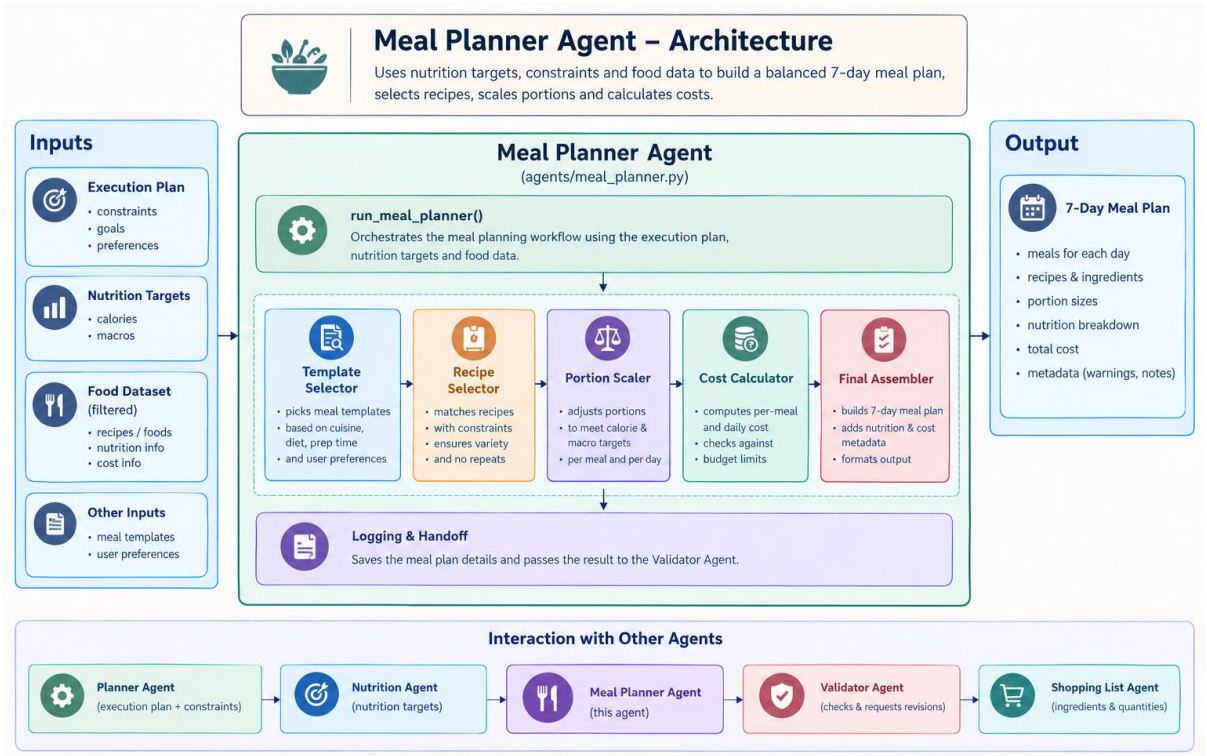
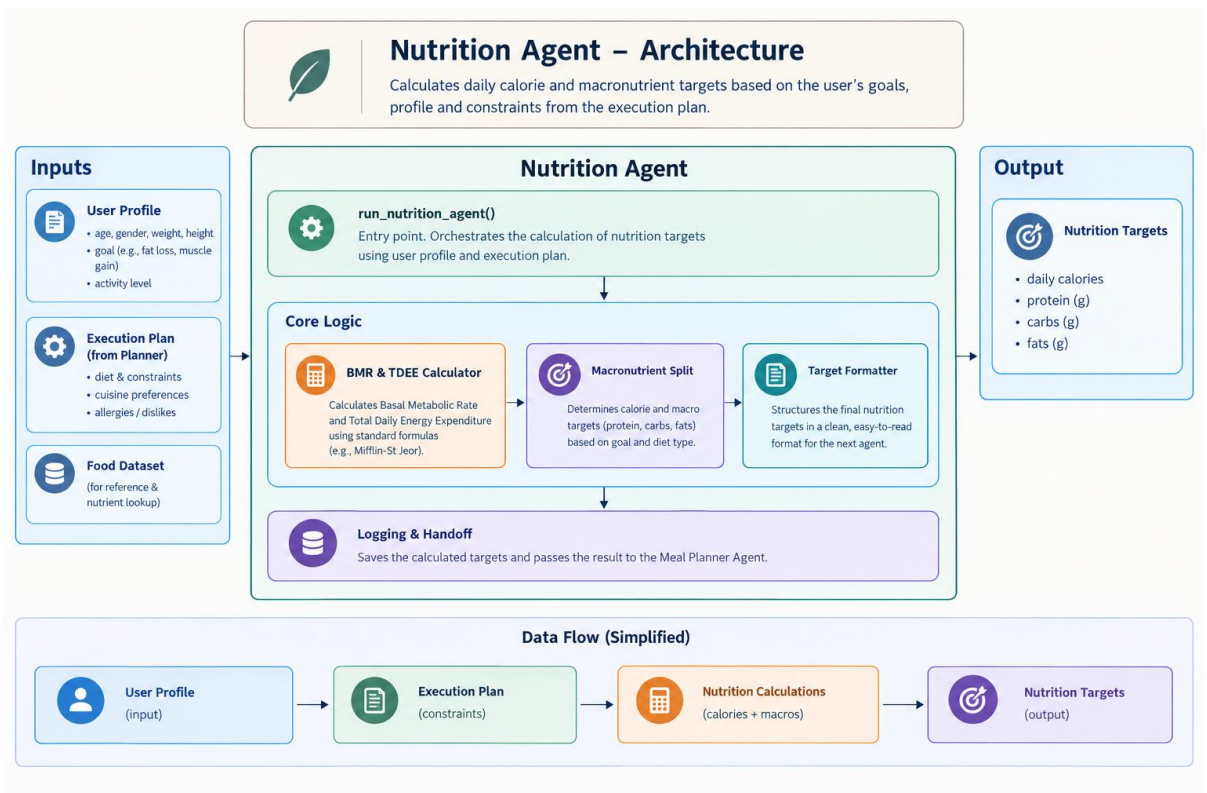
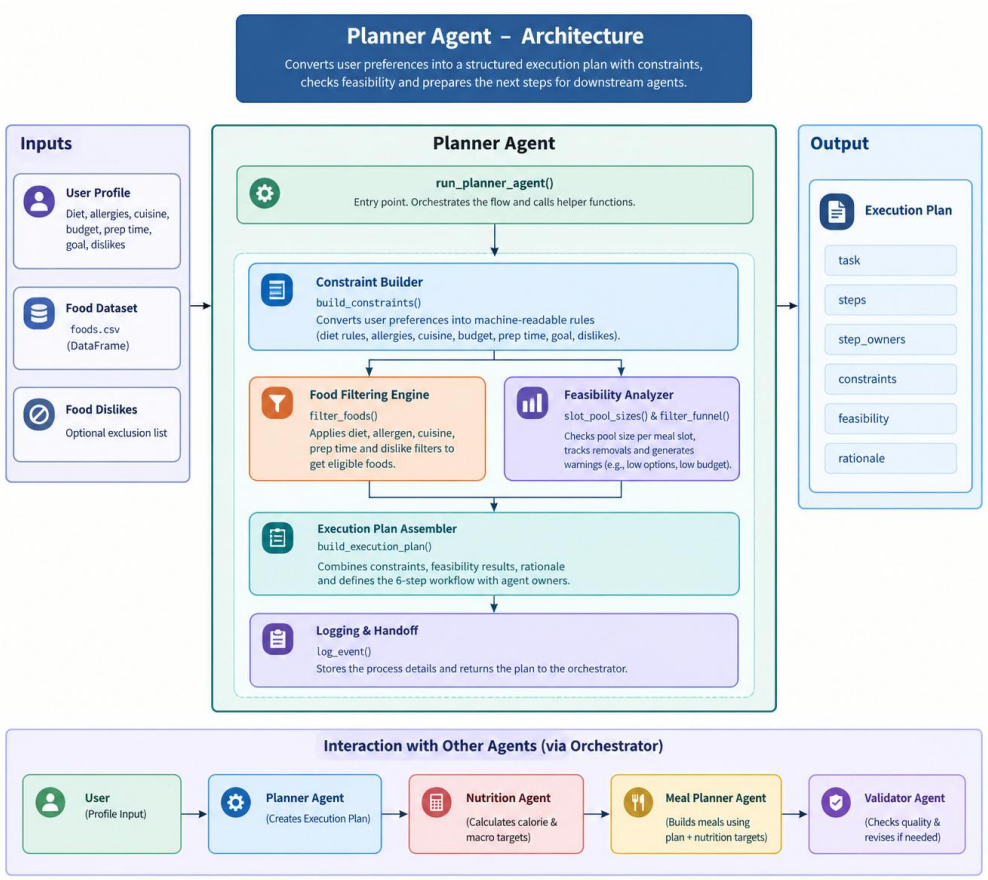
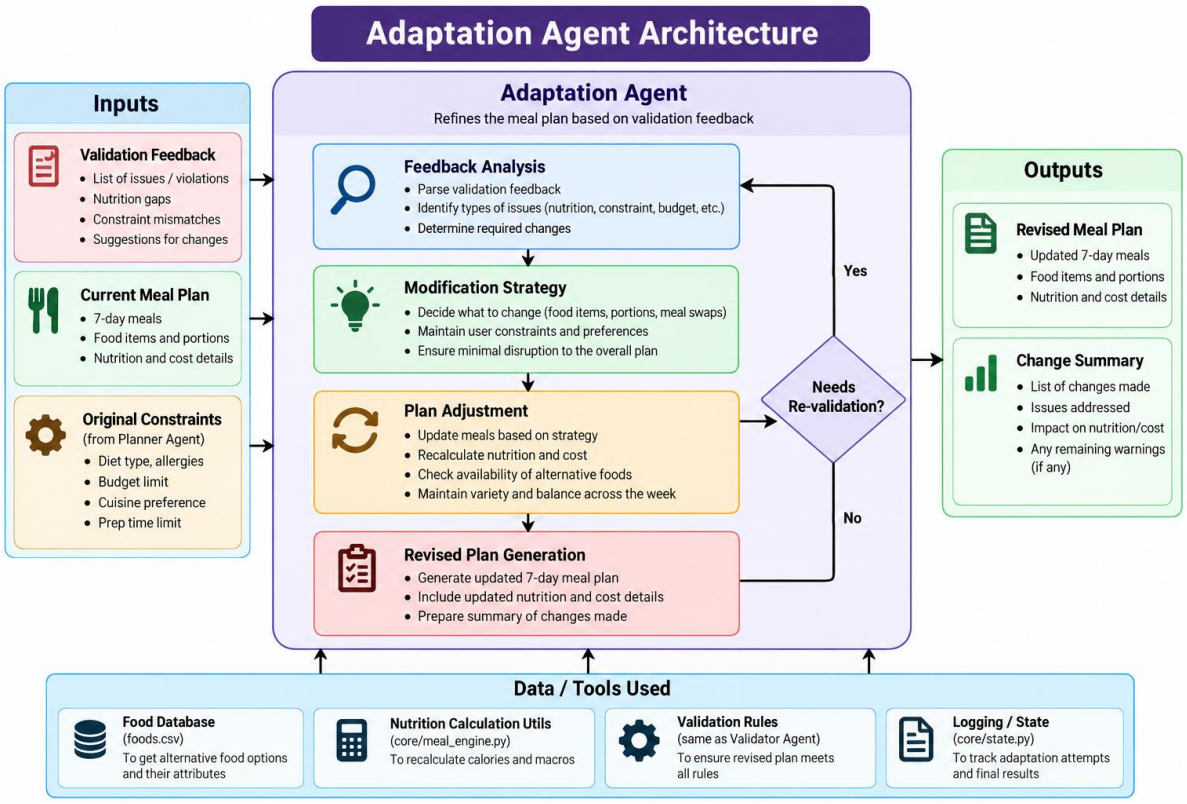
Architecture Diagram’s

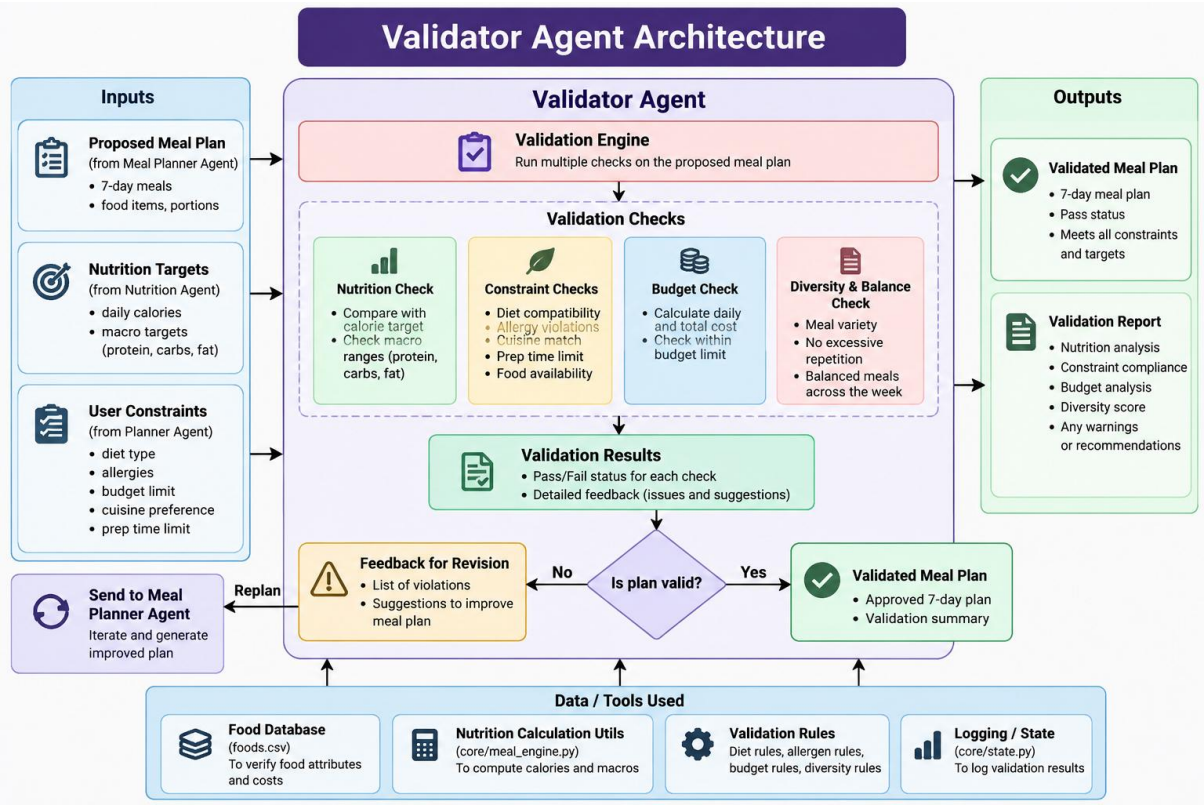



Meal Planner Agent – Architecture
Uses nutrition targets, constraints and food data to build a balanced 7-day meal plan, selects recipes, scales portions and calculates costs.
Inputs
Execution Plan
• constraints
• goals
• preferences
Nutrition Targets
• calories
• macros
Food Dataset (filtered)
• recipes / foods
• nutrition info
• cost info
Other Inputs
• meal templates
• user preferences
Meal Planner Agent
(agents/meal_planner.py)
run_meal_planner()
Orchestrates the meal planning workflow using the execution plan, nutrition targets and food data.
Template Selector
• picks meal templates
• based on cuisine, diet, prep time
• and user preferences
Recipe Selector
• matches recipes
• with constraints
• ensures variety
• and no repeats
Portion Scaler
• adjusts portions
• to meet calorie & macro targets
• per meal and per day
Cost Calculator
• computes per-meal and daily cost
• checks against
• budget limits
Final Assembler
• builds 7-day meal plan
• adds nutrition & cost metadata
• formats output
Logging & Handoff
Saves the meal plan details and passes the result to the Validator Agent.
Output
7-Day Meal Plan
• meals for each day
• recipes & ingredients
• portion sizes
• nutrition breakdown
• total cost
• metadata (warnings, notes)
Interaction with Other Agents
Planner Agent
(execution plan + constraints)
Nutrition Agent
(nutrition targets)
Meal Planner Agent
(this agent)
Validator Agent
(checks & requests revisions)
Shopping List Agent
(ingredients & quantities)


Nutrition Agent – Architecture
Calculates daily calorie and macronutrient targets based on the user's goals, profile and constraints from the execution plan.
Inputs
User Profile
• age, gender, weight, height
• goal (e.g., fat loss, muscle gain)
• activity level
Execution Plan (from Planner)
• diet & constraints
• cuisine preferences
• allergies / dislikes
Food Dataset
(for reference & nutrient lookup)
Nutrition Agent
run_nutrition_agent()
Entry point. Orchestrates the calculation of nutrition targets using user profile and execution plan.
Core Logic
BMR & TDEE Calculator
Calculates Basal Metabolic Rate and Total Daily Energy Expenditure using standard formulas (e.g., Mifflin-St Jeor).
Macronutrient Split
Determines calorie and macro targets (protein, carbs, fats) based on goal and diet type.
Target Formatter
Structures the final nutrition targets in a clean, easy-to-read format for the next agent.
Logging & Handoff
Saves the calculated targets and passes the result to the Meal Planner Agent.
Output
Nutrition Targets
• daily calories
• protein (g)
• carbs (g)
• fats (g)
Data Flow (Simplified)
User Profile
(input)
Execution Plan
(constraints)
Nutrition Calculations
(calories + macros)
Nutrition Targets
(output)


Planner Agent – Architecture
Converts user preferences into a structured execution plan with constraints, checks feasibility and prepares the next steps for downstream agents.
Inputs
User Profile
Diet, allergies, cuisine, budget, prep time, goal, dislikes
Food Dataset
foods.csv (DataFrame)
Food Dislikes
Optional exclusion list
Planner Agent
run_planner_agent()
Entry point. Orchestrates the flow and calls helper functions.
Constraint Builder
build_constraints()
Converts user preferences into machine-readable rules (diet rules, allergies, cuisine, budget, prep time, goal, dislikes).
Food Filtering Engine
filter_foods()
Applies diet, allergen, cuisine, prep time and dislike filters to get eligible foods.
Feasibility Analyzer
slot_pool_sizes() & filter_funnel()
Checks pool size per meal slot, tracks removals and generates warnings (e.g., low options, low budget).
Execution Plan Assembler
build_execution_plan()
Combines constraints, feasibility results, rationale and defines the 6-step workflow with agent owners.
Logging & Handoff
log_event()
Stores the process details and returns the plan to the orchestrator.
Output
Execution Plan
task
steps
step_owners
constraints
feasibility
rationale
Interaction with Other Agents (via Orchestrator)
User
(Profile Input)
Planner Agent
(Creates Execution Plan)
Nutrition Agent
(Calculates calorie & macro targets)
Meal Planner Agent
(Builds meals using plan + nutrition targets)
Validator Agent
(Checks quality & revises if needed)


Adaptation Agent Architecture
Inputs
Validation Feedback
List of issues / violations
Nutrition gaps
Constraint mismatches
Suggestions for changes
Current Meal Plan
7-day meals
Food items and portions
Nutrition and cost details
Original Constraints
(from Planner Agent)
Diet type, allergies
Budget limit
Cuisine preference
Prep time limit
Adaptation Agent
Refines the meal plan based on validation feedback
Feedback Analysis
Parse validation feedback
Identify types of issues (nutrition, constraint, budget, etc.)
Determine required changes
Modification Strategy
Decide what to change (food items, portions, meal swaps)
Maintain user constraints and preferences
Ensure minimal disruption to the overall plan
Plan Adjustment
Update meals based on strategy
Recalculate nutrition and cost
Check availability of alternative foods
Maintain variety and balance across the week
Revised Plan Generation
Generate updated 7-day meal plan
Include updated nutrition and cost details
Prepare summary of changes made
Needs Re-validation?
Yes
No
Outputs
Revised Meal Plan
Updated 7-day meals
Food items and portions
Nutrition and cost details
Change Summary
List of changes made
Issues addressed
Impact on nutrition/cost
Any remaining warnings (if any)
Data / Tools Used
Food Database
(foods.csv)
To get alternative food options and their attributes
Nutrition Calculation Utils
(core/meal_engine.py)
To recalculate calories and macros
Validation Rules
(same as Validator Agent)
To ensure revised plan meets all rules
Logging / State
(core/state.py)
To track adaptation attempts and final results


Validator Agent Architecture
Inputs
Proposed Meal Plan
(from Meal Planner Agent)
• 7-day meals
• food items, portions
Nutrition Targets
(from Nutrition Agent)
• daily calories
• macro targets (protein, carbs, fat)
User Constraints
(from Planner Agent)
• diet type
• allergies
• budget limit
• cuisine preference
• prep time limit
Validator Agent
Validation Engine
Run multiple checks on the proposed meal plan
Validation Checks
Nutrition Check
• Compare with calorie target
• Check macro ranges (protein, carbs, fat)
Constraint Checks
• Diet compatibility
• Allergy violations
• Cuisine match
• Prep time limit
• Food availability
Budget Check
• Calculate daily and total cost
• Check within budget limit
Diversity & Balance Check
• Meal variety
• No excessive repetition
• Balanced meals across the week
Validation Results
• Pass/Fail status for each check
• Detailed feedback (issues and suggestions)
Is plan valid?
No
Yes
Feedback for Revision
• List of violations
• Suggestions to improve meal plan
Validated Meal Plan
• Approved 7-day plan
• Validation summary
Replan
Send to Meal Planner Agent
Iterate and generate improved plan
Outputs
Validated Meal Plan
• 7-day meal plan
• Pass status
• Meets all constraints and targets
Validation Report
• Nutrition analysis
• Constraint compliance
• Budget analysis
• Diversity score
• Any warnings or recommendations
Data / Tools Used
Food Database
(foods.csv)
To verify food attributes and costs
Nutrition Calculation Utils
(core/meal_engine.py)
To compute calories and macros
Validation Rules
Diet rules, allergen rules, budget rules, diversity rules
Logging / State
(core/state.py)
To log validation results